

一、vLLM

《PagedAttention》(SOSP 2023)

解决什么问题：

大模型生成新词的时候，需要把前面已经算过的 token 的 key 和 value 存下来 (KV cache)，不然每生成一个词都要把前面全部重算一遍效率太低。之前给每个请求预先划好一整块连续显存放这些缓存，请求长度有长有短，划多了浪费、划少了不够用，导致显存里就出现了很多碎片，利用率很低。

怎么做：它把 KV cache 切成一个个固定大小的小块 (叫 page)，像操作系统里的虚拟内存分页那样，用到哪一块就分配哪一块，不再要求一整片连续的空间。

大概就是把“内存不够用时靠分页来省空间”的老办法搬到了显存上。原来是一口气占一块内存，现在改成一块一块地调用，这样同一张显卡就能同时服务更多的请求。

Question 具体多大是怎么定的、缓存满了之后怎么换进换出，如何实现这些细节

二、SGLang

《RadixAttention》(NeurIPS 2024)

解决什么问题：很多请求的开头是重复的，比如大家共用的一段系统提示词。如果每个请求都把这个相同的前缀从头算一遍，就白白浪费了算力。

怎么做：SGLang 用一个叫 RadixCache 的结构（底层是前缀树 radixtree）把 KVcache 按前缀组织起来，RadixAttention 会自动判断新来的请求和之前有没有相同的前缀，有的话就直接复用已经算好的缓存，这部分就叫 prefix reuse（前缀复用）。

question 前缀树在请求不断进来、又不断结束的情况下，是怎么维护和回收旧缓存的。